Exercise 3:

Karim Negm - 11063520

Exercise Question 1:

A LoRaWAN network in Europe (carrier frequency 868 MHz, bandwidth 125 kHz) is composed by one gateway and 50 sensor nodes. The sensor nodes transmit packet with payload size of L byte according to a Poisson process with intensity lambda = 1 packet / minute. Find the biggest LoRa SF for having a success rate of at least 70%.

ANSWER:

SF8

Answer motivation:

- To find the biggest LoRa Spreading Factor (SF) for a 50-node network (1 packet/min/node) needing >= 70% success rate with one gateway:
    1. **Model:** Use the Pure Aloha model (Success Rate = e^(-2G)), as nodes transmit randomly.
    2. **Load (G):** Calculate the total network traffic (50 nodes * 1/60 pkt/sec) multiplied by the packet's airtime (T_packet).
    3. **Airtime (T_packet):** Get the specific airtime (in seconds) for each SF (SF7 to SF12) with the given payload (23 bytes), BW (125kHz), and CR (4/5). *Higher SF means longer airtime.*
    4. **Calculate & Check:** For each SF, calculate G, then calculate the success rate. Check if it's >= 0.70.
    5. **Find Max SF:** Identify the highest SF number (starting from SF12 down) that meets the 70% threshold.
- The Python code automates steps 3-5.

Python Script:

```
import math
# --- Parameters ---
N_nodes = 50
lambda_node_per_sec = 1.0 / 60.0  # Packet rate per node in packets/second
DER_threshold = 0.70
# Airtime for each SF in milliseconds (from https://www.thethingsnetwork.org/airtime-calculator)
airtimes_ms = {
    7: 77.1,
    8: 143.9,
    9: 267.3,
    10: 493.6,
    11: 987.1,
    12: 1974.3
}
```

```
# --- Calculations ---

# Calculate total network traffic rate
lambda_total_per_sec = N_nodes * lambda_node_per_sec

print(f"Number of Nodes (N): {N_nodes}")
print(f"Total Network Traffic Rate (Lambda_total): {lambda_total_per_sec:.4f}
packets/second")
print(f"Required Success Rate (DER): >= {DER_threshold:.2f}")
print("-" * 30)

valid_sfs = []

# Iterate through Spreading Factors
# Sorting keys ensures we process from SF7 to SF12
for sf in airtimes_ms.keys():
    # Get airtime and convert to seconds
    t_packet_ms = airtimes_ms[sf]
    t_packet_s = t_packet_ms / 1000.0

    # Calculate offered load (G)
    G = lambda_total_per_sec * t_packet_s

    # Calculate success rate (DER) using Aloha formula
    DER = math.exp(-2 * G)

    print(f"SF{sf}:")
    print(f"  Airtime (T_packet) = {t_packet_ms:.1f} ms = {t_packet_s:.4f} s")
    print(f"  Offered Load (G)   = {G:.4f}")
    print(f"  Calculated DER     = {DER:.4f}")

    # Check if DER meets the threshold
    if DER >= DER_threshold:
        print(f"  ---> Meets threshold (>= {DER_threshold:.2f})")
        valid_sfs.append(sf)
    else:
        print(f"  ---> Does NOT meet threshold")
    print("-" * 10)

print("=" * 30)

# Determine the biggest valid SF
if not valid_sfs:
    print("Result: No Spreading Factor meets the 70% success rate requirement.")
else:
    biggest_valid_sf = max(valid_sfs)
    print(f"Result: Spreading Factors meeting the requirement: {valid_sfs}")
    print(f"The biggest LoRa SF for having a success rate of at least 70% is:
SF{biggest_valid_sf}")
print("=" * 30)
```

Output:

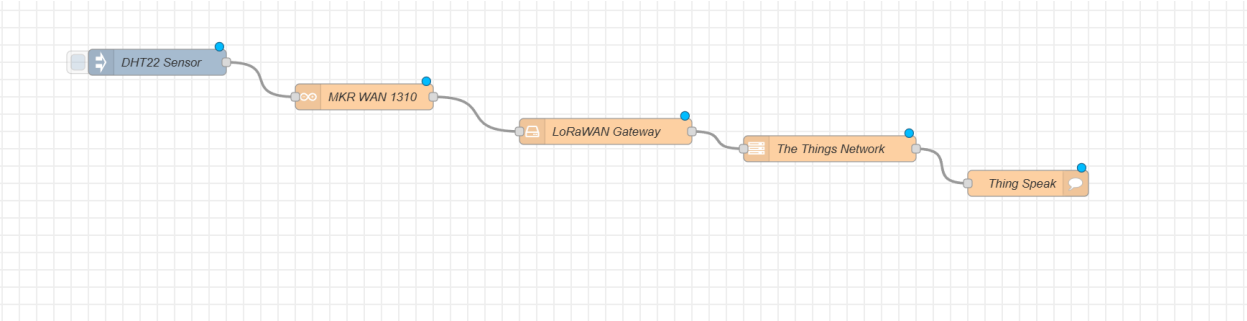
Result: Spreading Factors meeting the requirement: [7, 8]

The biggest LoRa SF for having a success rate of at least 70% is: SF8

Exercise Question 2:

You have purchased an Arduino MKR WAN 1310 and wish to create a system that reads temperature and humidity data from a DHT22 sensor and sends this data wirelessly to ThingSpeak over LoRaWAN. Design a complete system block diagram (sketch in Node-Red) and describe, in detail, the steps you would need to take to get the system fully operational

ANSWER:



Goal:

- Show temperature/humidity from a DHT22 sensor on a ThingSpeak web graph, using LoRaWAN for wireless transmission.

Main Parts:

- DHT22: Gets the temp/humidity readings.
- Arduino MKR WAN 1310: Reads the DHT22, sends data using its LoRaWAN radio.
- LoRaWAN Gateway: Receives the radio signal, forwards it to the internet.
- The Things Network (TTN): Cloud service that understands the LoRaWAN message and forwards the data.
- ThingSpeak: Cloud service that receives the final data and makes the graphs.

How it Works:

1. **Connect:** Wire the DHT22 sensor to the Arduino MKR WAN.
2. **Register:**
   a. Get the unique **DevEUI** ID from the Arduino board.
   b. Tell TTN about your device using this DevEUI. TTN gives you back **AppEUI** and **AppKey** keys.
3. **Program Arduino:**
   a. Write code to:
   b. Use the TTN keys (AppEUI, AppKey, DevEUI) to join the LoRaWAN network (OTAA).
   c. Read temperature/humidity from the DHT22.
   d. **Important:** Convert the readings into a small package of **bytes** (not text).

e. Send these bytes using the LoRaWAN radio.
f. Wait/sleep for several minutes.

4. **Configure TTN:**

   a. **Decoder:** Provide a Javascript function on TTN to translate the **bytes** received from the Arduino back into numbers
   b. **Webhook:** Set up a ThingSpeak webhook on TTN. Give it your ThingSpeak Channel ID and Write API Key. Tell it which decoded value (e.g., "temperature") goes to which ThingSpeak field (e.g., field1).

5. **Configure ThingSpeak:**

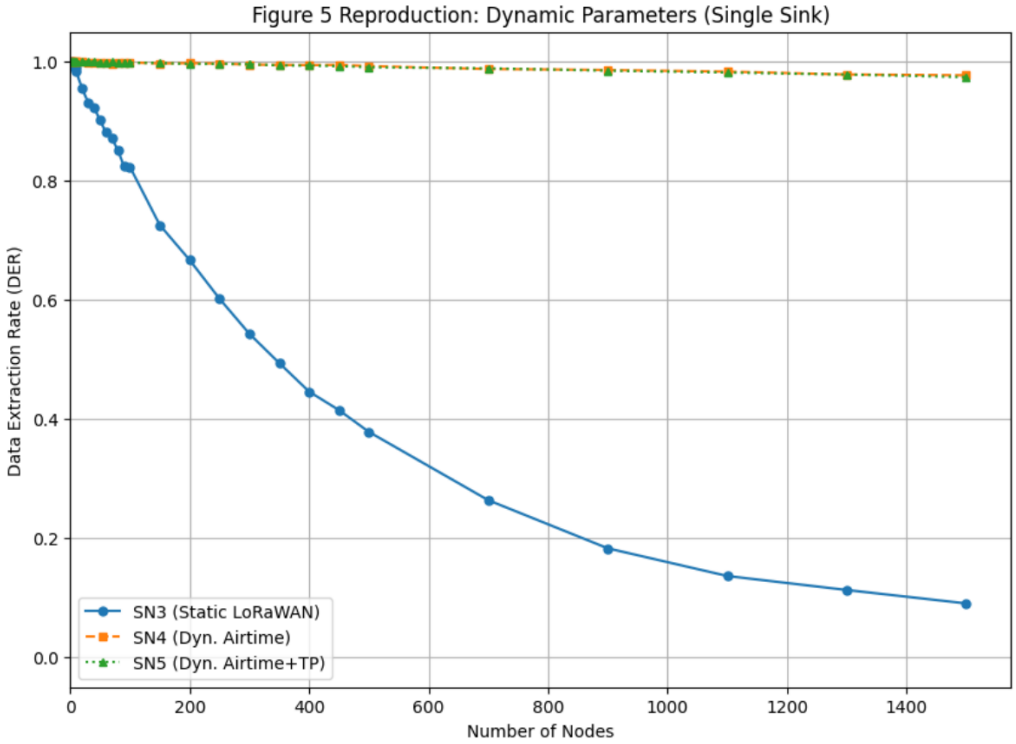
   a. Create a channel.
   b. Enable fields (e.g., Field 1 for Temp, Field 2 for Hum).
   c. Get the Channel ID and Write API Key (for the TTN webhook).
   d. Make the channel **Public** (Sharing tab).

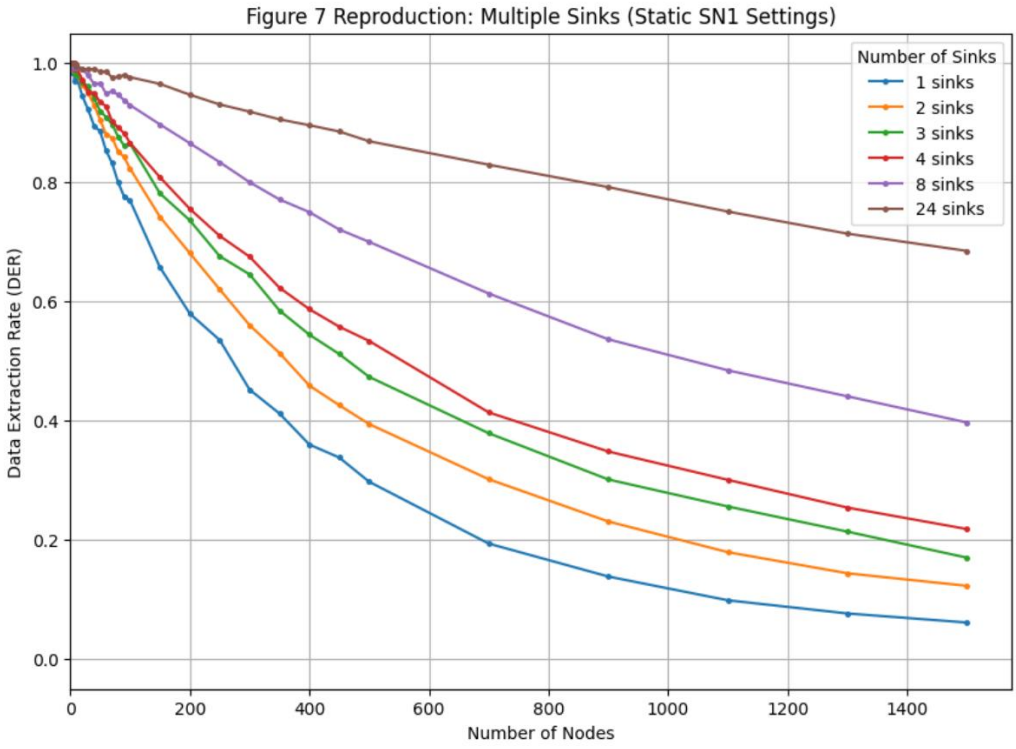
6. **Done:** Data flows from Arduino -> Gateway -> TTN (decodes) -> ThingSpeak (graphs).

Exercise Question 3:

Using the paper “Do LoRa Low-Power Wide-Area Networks Scale?” by M. Bor et al. and the LoRa simulator available at LoRaSim , your task is to reproduce Figure 5 and Figure 7 from the paper

Results:





Answer motivation:

1. Objective:

The task was to reproduce Figures 5 and 7 using the supplied LoRaSim simulator via the provided Jupyter Notebook (LoRasim.ipynb).

Figure 5 shows the effect of dynamic parameter settings versus static settings (single sink).

Figure 7 shows the impact of multiple sinks (using static settings).

2. Methodology:

The provided notebook initially only simulated the scenario for Figure 4. To achieve the objective, the notebook was modified to orchestrate the execution of the correct LoRaSim Python 2 scripts with the required parameters:

- **Unified Simulation Control:** A Python function (simulate) was defined within the notebook to call the external simulation scripts (loraDir.py for single-sink and loraDirMulBS.py for multi-sink scenarios) using subprocess.run, passing the correct command-line arguments (node count, experiment number, duration, base station count, collision model).
- **Reproducing Figure 5:**
    - The simulate function was called repeatedly, targeting the loraDir.py script (single sink).
    - Simulations were run for **experiments 4, 3, and 5** (representing static, dynamic airtime, and dynamic airtime+power settings, respectively) across a range of node counts.
    - The resulting output files (exp4.dat, exp3.dat, exp5.dat) were processed using pandas to calculate the Data Extraction Rate (DER) from the collision and transmission counts.
    - The calculated DER values were plotted against the number of nodes using matplotlib to generate the plot corresponding to Figure 5.
- **Reproducing Figure 7:**
    - The simulate function was called repeatedly, this time targeting the loraDirMulBS.py script (multi-sink).
    - Simulations were run using only **experiment 0** (representing the static SN1 settings used in the paper's Figure 7).
    - The number of base stations was varied (**1, 2, 3, 4, 8, 24**) for different simulation runs across a range of total node counts.
    - The output files (e.g., exp0BS1.dat, exp0BS8.dat, etc.), which contain pre-calculated DER, were read using pandas.

- The DER values were plotted against the number of nodes using matplotlib, with a separate line for each base station count, generating the plot corresponding to Figure 7.

3. Outcome:

By modifying the Jupyter Notebook to correctly call the appropriate LoRaSim scripts with the necessary parameters and processing the output data, plots visually corresponding to Figure 5 and Figure 7 from the Bor et al. paper were successfully generated, demonstrating the simulated effects of dynamic parameters and multiple sinks on LoRa network scalability.

1. Setup the Environment

This cell downloads LoRaSim, installs Python 2, and the necessary dependencies. Run this cell first.

In [1]:
```
# ! wget https://www.lancaster.ac.uk/scc/sites/lora/lorasim-20170710.tgz -O lorasim-20170710.tgz
# ! tar -xzf lorasim-20170710.tgz # Use -xzf for .tgz

# # Install Python 2 and pip for Python 2
# !sudo apt-get update -q
# !sudo apt-get install python2 python2-dev -q
# !curl https://bootstrap.pypa.io/pip/2.7/get-pip.py -o get-pip.py
# !sudo python2 get-pip.py
# !rm get-pip.py

# # Install Python Tkinter for matplotlib backend (if needed, though Agg should avoid it)
# !sudo apt-get install python-tk -q

# # Install dependencies using pip for Python 2
# !pip2 install -r 'lorasim/requirements.txt' --ignore-installed six # Added --ignore-installed six for potential conflicts

# print("\n--- Environment Setup Complete ---")
```

2. Import Libraries and Define Simulation Functions

Import standard libraries and define the core `simulate` function capable of calling both LoRaSim scripts, along with the `aloha_der` function.

In [2]:
```
import os
import pandas as pd
import math
import subprocess
import matplotlib
import matplotlib.pyplot as plt

print(f"Using Matplotlib version: {matplotlib.__version__}") # Check matplotlib version

# Store original MPLBACKEND if it exists
```

```python
original_mplbackend = os.environ.get("MPLBACKEND", None)

def simulate(n_nodes, tx_rate, exp, duration, nrBS=1, use_multi_bs_script=False, collision=True):
    """
    Runs the LoRaSim simulation.

    Args:
        n_nodes (int): Number of nodes (total nodes if multi-bs).
        tx_rate (float): Average send interval (ms).
        exp (int): Experiment number (0-5 for loraDir, 0-4 for loraDirMulBS).
        duration (int): Simulation time (ms).
        nrBS (int): Number of base stations (used only if use_multi_bs_script is True).
        use_multi_bs_script (bool): If True, calls loraDirMulBS.py, otherwise loraDir.py.
        collision (bool): Whether to use full collision check (True) or simplified (False).
    """
    env = os.environ.copy()
    # Set MPLBACKEND to Agg to prevent Tkinter issues in non-GUI environments
    env["MPLBACKEND"] = "Agg"

    command = ["python2"]
    script_to_run = "lorasim/loraDirMulBS.py" if use_multi_bs_script else "lorasim/loraDir.py"
    command.append(script_to_run)

    command.extend([
        str(int(n_nodes)),
        str(int(tx_rate)),
        str(int(exp)),
        str(int(duration)),
    ])

    if use_multi_bs_script:
        command.append(str(int(nrBS)))

    # Add collision argument - assumes it's the last optional argument for both scripts
    command.append("1" if collision else "0")

    print(f"Running command: {' '.join(command)}") # Debug print

    try:
        result = subprocess.run(
            command,
            env=env,
```

```
            capture_output=True,
            text=True,  # Capture output as text
            check=False # Don't raise exception on non-zero exit code immediately
        )

        # Print stdout/stderr for debugging purposes
        # print("STDOUT:\n", result.stdout)
        # print("STDERR:\n", result.stderr)


        if result.returncode != 0:
             print(f"Warning: Simulation command exited with code {result.returncode}")
             print(f"STDERR for error check:\n{result.stderr[:500]}...") # Print beginning of stderr
             # Optionally, decide if this error should stop the process
             # raise Exception(f"Simulation failed: {result.stderr}")

    except FileNotFoundError:
        print(f"Error: Python 2 executable not found or script '{script_to_run}' missing.")
        raise # Re-raise the error to stop execution
    except Exception as e:
        print(f"An unexpected error occurred running the simulation: {e}")
        raise # Re-raise

    finally:
      # Restore original MPLBACKEND if it existed
      if original_mplbackend:
          os.environ["MPLBACKEND"] = original_mplbackend
      elif "MPLBACKEND" in os.environ:
          # If it wasn't set before, remove the temporary setting
          del os.environ["MPLBACKEND"]


# Define aloha_der function for theoretical comparison
def aloha_der(n_nodes, t, packet_rate_per_node=1e-6):
    """
    Calculates theoretical Aloha DER (S/G = e^-2G).

    Args:
        n_nodes (int): Number of nodes.
        t (float): Packet airtime in milliseconds.
        packet_rate_per_node (float): Average packets per millisecond per node.
    """
```

```
    # G = N * lambda * t (lambda is packet rate per node, t is airtime)
    # Ensure t is treated as milliseconds if rate is per ms
    G = n_nodes * packet_rate_per_node * t
    return math.exp(-2 * G)

print("--- Helper Functions Defined ---")
```

```
Using Matplotlib version: 3.7.5
--- Helper Functions Defined ---
```

3. Run Simulations

Execute the simulation loops required for both Figure 5 and Figure 7. This will take a considerable amount of time.

In [3]:
```
# --- Simulation Parameters ---
duration = 86400000 # 24 hours in ms
tx_rate = 1e6      # 1 packet per 1,000,000 ms (approx. 16.7 min average interval)
collision_check = True # Use full collision model for both figures

# Node counts to simulate (adjust range/step as needed for time vs resolution)
nodes_list = list(range(1,10, 2)) + list(range(10,100,10)) + list(range(100, 500, 50)) + list(range(500, 1700, 200))
# nodes_list = list(range(10, 110, 10)) # Faster list for testing

# --- Run Simulations for Figure 5 (Dynamic Parameters, Single Sink) ---
print("\n--- Running Simulations for Figure 5 ---")
exps_fig5 = [4, 3, 5] # SN3 (static), SN4 (dyn airtime), SN5 (dyn airtime+TP)
for exp_fig5 in exps_fig5:
    print(f"\nRunning experiment {exp_fig5} for Figure 5...")
    # Clear previous file for this experiment
    fname = f"exp{exp_fig5}.dat"
    if os.path.exists(fname):
        try: os.remove(fname)
        except OSError as e: print(f"Error removing {fname}: {e}")

    for n_nodes in nodes_list:
      print(f"  Simulating {n_nodes} nodes (Exp {exp_fig5})...")
      simulate(n_nodes, tx_rate, exp_fig5, duration, nrBS=1, use_multi_bs_script=False, collision=collision_check)
print("--- Figure 5 Simulations Complete ---")


# --- Run Simulations for Figure 7 (Multiple Sinks, Static SN1) ---
```

```
print("\n--- Running Simulations for Figure 7 ---")
exp_fig7 = 0 # Experiment 0 corresponds to SN1 used in Fig 7
base_station_counts_fig7 = [1, 2, 3, 4, 8, 24] # As shown in Figure 7
for bs_count in base_station_counts_fig7:
    print(f"\nRunning experiment {exp_fig7} for Figure 7 with {bs_count} Base Station(s)...")
    # Clear previous file for this BS count
    fname = f"exp{exp_fig7}BS{bs_count}.dat"
    if os.path.exists(fname):
       try: os.remove(fname)
       except OSError as e: print(f"Error removing {fname}: {e}")

    for n_nodes in nodes_list:
        # Important: n_nodes for loraDirMulBS.py is TOTAL nodes
        print(f"  Simulating {n_nodes} total nodes ({bs_count} BS)...")
        simulate(n_nodes, tx_rate, exp_fig7, duration, nrBS=bs_count, use_multi_bs_script=True, collision=collision_check)
print("--- Figure 7 Simulations Complete ---")
```

```
STDERR for error check:
...
  Simulating 900 total nodes (24 BS)...
Running command: python2 lorasim/loraDirMulBS.py 900 1000000 0 86400000 24 1
Warning: Simulation command exited with code 255
STDERR for error check:
...
  Simulating 1100 total nodes (24 BS)...
Running command: python2 lorasim/loraDirMulBS.py 1100 1000000 0 86400000 24 1
Warning: Simulation command exited with code 255
STDERR for error check:
...
  Simulating 1300 total nodes (24 BS)...
Running command: python2 lorasim/loraDirMulBS.py 1300 1000000 0 86400000 24 1
Warning: Simulation command exited with code 255
STDERR for error check:
...
  Simulating 1500 total nodes (24 BS)...
Running command: python2 lorasim/loraDirMulBS.py 1500 1000000 0 86400000 24 1
Warning: Simulation command exited with code 255
STDERR for error check:
...
--- Figure 7 Simulations Complete ---
```

4. Process Results and Generate Plots

Load the `.dat` files generated by the simulations, calculate Data Extraction Rate (DER), and plot the results to reproduce Figure 5 and Figure 7.

In [4]:

```
print("\n--- Processing and Plotting Results ---")

# --- Process and Plot Figure 5 ---
plt.figure(figsize=(10, 7))
print("Processing Figure 5 data...")
exps_fig5 = [4, 3, 5]
labels_fig5 = {4: "SN3 (Static LoRaWAN)", 3: "SN4 (Dyn. Airtime)", 5: "SN5 (Dyn. Airtime+TP)"}
markers = {4: 'o', 3: 's', 5: '^'}
linestyles = {4: '-', 3: '--', 5: ':'}

fig5_data_loaded = False
for exp_fig5 in exps_fig5:
```

```
    fname = f"exp{exp_fig5}.dat"
    try:
        # loraDir.py output: #nrNodes nrCollisions nrTransmissions OverallEnergy
        df = pd.read_csv(fname, sep=" ", comment='#', names=["#nrNodes", "nrCollisions", "nrTransmissions", "OverallEnergy"], header=None, skip
        # Ensure transmissions > 0 to avoid division by zero
        df = df[df["nrTransmissions"] > 0].copy()
        # Calculate DER = Received / Transmitted = (Transmitted - Collisions) / Transmitted
        # Note: nrCollisions might count packets involved in *any* collision, not just lost ones.
        # The paper defines DER based on successfully received packets. The script output might need careful interpretation.
        # Let's assume (Transmitted - Collisions) is a proxy for received for now, based on loraDir.py logic.
        df["der"] = (df["nrTransmissions"] - df["nrCollisions"]) / df["nrTransmissions"]
        # Ensure DER is not negative (can happen if collisions exceed transmissions due to error/counting method)
        df["der"] = df["der"].clip(lower=0)

        # Check for empty dataframe after filtering
        if df.empty:
            print(f"  Warning: No data with transmissions > 0 found in {fname}. Skipping plot for this experiment.")
            continue

        plt.plot(df["#nrNodes"], df["der"], label=labels_fig5[exp_fig5], marker=markers[exp_fig5], linestyle=linestyles[exp_fig5], markersize=5
        fig5_data_loaded = True
        print(f"Loaded and processed {fname}")

    except FileNotFoundError:
        print(f"  Warning: Data file not found for Figure 5, experiment {exp_fig5} ({fname}). Skipping plot for this experiment.")
    except pd.errors.EmptyDataError:
         print(f"  Warning: Data file {fname} is empty. Skipping plot for this experiment.")
    except Exception as e:
        print(f"  Error processing file {fname}: {e}")

if fig5_data_loaded:
    plt.title("Figure 5 Reproduction: Dynamic Parameters (Single Sink)")
    plt.xlabel("Number of Nodes")
    plt.ylabel("Data Extraction Rate (DER)")
    plt.ylim(-0.05, 1.05)
    plt.xlim(left=0)
    plt.legend()
    plt.grid(True)
    plt.show()
else:
    print("No data successfully loaded for Figure 5, plot skipped.")
```

```python
# --- Process and Plot Figure 7 ---
plt.figure(figsize=(10, 7))
print("\nProcessing Figure 7 data...")
exp_fig7 = 0 # SN1 settings
base_station_counts_fig7 = [1, 2, 3, 4, 8, 24]
fig7_data_loaded = False

for bs_count in base_station_counts_fig7:
    fname = f"exp{exp_fig7}BS{bs_count}.dat"
    try:
        # loraDirMulBS.py output: # nrNodes DER (DER is already calculated)
        df = pd.read_csv(fname, sep=" ", comment='#', names=["#nrNodes", "DER"], header=None, skipinitialspace=True)
        # Ensure DER is not negative
        df["DER"] = df["DER"].clip(lower=0)

        # Check for empty dataframe
        if df.empty:
            print(f"  Warning: No data found in {fname}. Skipping plot for this configuration.")
            continue

        plt.plot(df["#nrNodes"], df["DER"], label=f"{bs_count} sinks", marker='.', linestyle='-', markersize=5)
        fig7_data_loaded = True
        print(f"Loaded and processed {fname}")

    except FileNotFoundError:
         print(f"  Warning: Data file not found for Figure 7, {bs_count} base stations ({fname}). Skipping plot for this configuration.")
    except pd.errors.EmptyDataError:
         print(f"  Warning: Data file {fname} is empty. Skipping plot for this configuration.")
    except Exception as e:
        print(f"  Error processing file {fname}: {e}")


if fig7_data_loaded:
    plt.title("Figure 7 Reproduction: Multiple Sinks (Static SN1 Settings)")
    plt.xlabel("Number of Nodes")
    plt.ylabel("Data Extraction Rate (DER)")
    plt.ylim(-0.05, 1.05)
    plt.xlim(left=0)
    plt.legend(title="Number of Sinks")
    plt.grid(True)
    plt.show()
```